Cost-Minimal Parameter Correction Subsets for Unsatisfiable Constraint Problems*

Antoine Laviolette[1][0009−0008−7153−2502],
Claude-Guy Quimper[1][0000−0002−5899−0217], and
Raphaël Boudreault[2][0000−0002−5602−7515]

[1] Université Laval, Québec, Canada
[2] Thales, Québec, Canada
Antoine.Laviolette.1@ulaval.ca, Claude-Guy.Quimper@ift.ulaval.ca,
Raphael.Boudreault@thalesgroup.com

Abstract. Decision support systems developed for real-world applications often rely on solving complex combinatorial problems using carefully crafted constraint models. However, users of such systems may input data that leads to a problem without any solution, rendering it unsatisfiable. The responsibility of understanding and correctly modifying the data generally falls on the user. Previous research on unsatisfiability mainly focused on reporting subsets of problematic constraints, which requires a good understanding of the underlying model. We introduce the concept of *Cost-Minimal Parameter Correction Subset* (CMPCS) as a subset of data parameters and their correction to make an instance satisfiable at a minimal cost. We propose a problem-independent approach, called *CMPCS-Finder*, which aims at reporting a CMPCS given an unsatisfiable instance. We empirically evaluate this approach on existing combinatorial benchmark problems and demonstrate that, in most cases, it successfully identifies the changes in the data that initially caused the instances to be unsatisfiable.

Keywords: Combinatorial Optimization · Unsatisfiability · Constraints · Correction · Modeling

1 Introduction

Decision support systems developed for real-world applications often rely on solving complex combinatorial problems and are usually designed as follows. A carefully crafted constraint model is used in the background and depends on specific parameters as input. Each time a user interacts with the system, new parameters are provided, but the underlying model remains unchanged. For instance, a scheduling system would have constraints in the model to ensure that resources are not overloaded and that tasks are given sufficient time to execute while meeting their deadlines. Here, the user data would contain the list of resources, as well as the tasks and their durations, deadlines, and resource

* This project has received financial support from the Mitacs Accelerate program.

consumption. However, during the use of such a system, it may happen that the combination model/data does not allow for any solution. Since the model is known to have been extensively tested and used for a long period of time with various datasets, it becomes clear that the unsatisfiability comes from the user data and not from the model.

The responsibility of understanding and correctly modifying the data generally falls on the user. To facilitate their task, it is possible to automatize some sanity checks, such as assertions, on the data. For example, one can easily detect if a task has a release time after its deadline. However, in other cases, the source of unsatisfiability can be much more complex due to the incompatibility between a large set of constraints and their associated parameters.

Previous research on explaining the unsatisfiability of such systems mainly focuses on reporting subsets of problematic constraints, which requires a good understanding of the underlying model [7, 9, 13]. Some approaches address only specific problems [1, 22], while others target particular types of constraints [10, 27, 28].

Thus, this paper aims to provide a new problem-independent approach, *CMPCS-Finder*, to detect inconsistencies in the data of a presumed correct constraint model given an unsatisfiable instance. For this, we introduce the concept of *Cost-Minimal Parameter Correction Subset* (CMPCS) as a subset of data parameters and their corrections to make an instance satisfiable at minimal cost. The cost of this subset of corrections can be measured by the number of parameters that need to be changed in the input or by any function that expresses the amount of effort required to change the input. Our proposed approach has the advantage of reporting a CMPCS interpretable by a user of such a system (e.g., a project manager using a scheduling system) without requiring any knowledge of the model.

The key contributions of our approach are:

- To propose a data-driven problem-independent method;
- To provide minimal corrections that are understandable to a user with little knowledge of the constraint model;
- To provide a system supporting a wide range of non-linear constraint types.

The paper is structured as follows. We first present the required background and related work on explaining the unsatisfiability of constraint problems. Then, we propose a methodology, *CMPCS-Finder*, to find a CMPCS, while detailing techniques to reduce its computational complexity. We empirically evaluate *CMPCS-Finder* on three existing combinatorial benchmark problems: the *Resource-Constrained Project Scheduling Problem* (RCPSP), the *Tank Allocation Problem* (TAP), and the *Roster Shift Bool Problem* (RSBP). Finally, we conclude and suggest directions for future research.

2 Background and Related Work

A *Constraint Satisfaction Problem* (CSP) is defined by a set of *variables* $\mathcal{X}$, where each variable $X \in \mathcal{X}$ is associated with a set of potential values $\mathrm{dom}(X)$

that form its *domain*. The variables are subject to a set of *constraints* $\mathcal{C}$ that encode the relationships between the variables and restrict their possible assignments. A subset of constraints $S \subseteq \mathcal{C}$ is said to be *satisfiable* if there exists a possible assignment of variables to values in their domains that satisfies all constraints $C \in S$. We say that S is *unsatisfiable* if no such assignment exists. A *solution* to a CSP is any assignment that makes $\mathcal{C}$ satisfiable. A *Constraint Optimization Problem* (COP) is a CSP where a special variable O, called the *objective*, is to be minimized. An *optimal* solution of a COP is a solution that minimizes O. In the following, we refer more generally to CSPs and COPs as *constraint problems*.

The *model* of a problem is the parameterized definition of a constraint problem in a logical (or mathematical) formulation. The parameters $\mathcal{P}$, which are usually numbers or sets, may be used in the definition of the constraints, e.g., as the coefficients of a linear equation. A constraint C is associated with a (possibly empty) parameter set $P(C) \subseteq \mathcal{P}$ such that changing the values of the parameters in $P(C)$ changes the solution space of the constraint C. A parameter may be associated with multiple constraints. The assignment of the model parameters to specific values defines an *instance* of the problem. In practice, the parameters are computed from the data provided by the user as input. High-level modeling languages, such as MiniZinc [19], allow models to be carefully designed using solver-independent declarative elements. Once the parameters are specified, the instances are compiled and given to the chosen solver.

The explanation of unsatisfiability has been extensively studied since the 1980s through the identification of *Minimal Unsatisfiable Subsets* (MUS) and *Minimal Correction Subsets* (MCS) [16]. A subset of constraints $M \subseteq \mathcal{C}$ is a MUS if and only if M is unsatisfiable and M' is satisfiable $\forall M' \subset M$. In other words, a MUS contains only the *core* constraints explaining the unsatisfiability. It is minimal in the sense of set inclusion. In operations research, these sets are sometimes called *Irreducibly/Minimally Inconsistent/Infeasible Subsystems* [31], while in Boolean satisfiability, they are referred to *Minimal Unsatisfiable Cores* [18]. Note that an unsatisfiable problem may admit more than one MUS. A subset of constraints $M \subseteq \mathcal{C}$ is an MCS if and only if $\mathcal{C} \setminus M$ is satisfiable and $\mathcal{C} \setminus M'$ is unsatisfiable $\forall M' \subset M$. In other words, an MCS forms a set of constraints to remove from $\mathcal{C}$ to *correct* it; that is, to make it satisfiable. It is also minimal in the sense of inclusion. Among the most common MUS/MCS extraction and enumeration techniques are QuickXplain [9] and MARCO [15, 16]. Detailed surveys of other techniques were made by Gupta *et al.* [6] and Marques-Silva *et al.* [17].

The work of Leo *et al.* [14] led to the development and integration of findMUS into MiniZinc, a debugging tool based on the extraction and enumeration of MUSes. Based on the previous ideas of Jussien *et al.* [10] on grouping constraints and hierarchical modeling, findMUS can identify a MUS regardless of the chosen solver while leveraging the hierarchical structure of MiniZinc constraints to speed up the process. This acceleration is particularly significant for the computation of a single MUS, which is sufficient in practice to independently correct the

model for each conflict. This approach thus allows for effectively identifying conflicts in the constraint model, but requires knowledge of the model to be interpretable by a user. Another related tool is MiniBrass [26], a soft constraint modeling language that allows solving problems with constraint *preferences*. This can be useful in always guaranteeing a solution to the problem for the user while requiring only small changes to the original model. On the other hand, this approach may ignore the constraint conflicts that come from the user-provided data. Furthermore, MiniBrass is limited to specific constraint types.

Senthooran *et al.* [27, 28] designed an industrial optimization system that can interactively report constraint conflicts and suggest corrections based on the computation of a MUS and an MCS. Their approach uses MiniZinc and findMUS, where annotations in the model describe constraints in order to link them to the user interface. The proposed corrections presented to the users relax a subset of constraints forming an MCS. Additionally, they add slack variables to relax the constraints in the MCS instead of removing the entire constraints. This helps make the corrections of the unsatisfiability less disruptive. Each constraint uses their own slack variable. This means that two constraints using the same parameter will have independent slack variables. This can be useful for obtaining fine-grained corrections, but it might not be applicable in some real-world scenarios. A limitation of their system is that it assumes the model is directly known and understandable by the users. Furthermore, only linear inequalities are supported in the models. Therefore, if a global constraint is used, Senthooran *et al.*'s approach will decompose it into linear inequalities and might return a subset of constraints from the decomposition that is unknown to the user.

Lauffer and Topcu [13] designed an unsatisfiability explanation system dedicated to resource-constrained scheduling problems. This system computes MUSes which can be represented either as a set of constraints or as a set of related tasks. For frequently occurring MUSes, developers can also craft specific messages to help users understand the unsatisfiability. Similarly, Poveda *et al.* [22] propose an interactive system tailored for workforce allocation, where users can select the right set of tasks to omit to make the problem satisfiable.

Bogaerts *et al.* [2] developed a generic approach to generate explanations as sequences of simple steps. Their method constructs explanations iteratively by leveraging the inference steps performed by the solver during the resolution process, while a cost function evaluates the simplicity of the explanations. The main challenge lies in effectively using the solver's information to provide clear and accessible explanations to the users. Subsequent improvements were proposed by Bleukx *et al.* [1] and Gamba *et al.* [7] to simplify the sequences, notably by filtering out unnecessary steps. While this approach has the advantage of explaining the unsatisfiability to the user, it does not provide a correction to make the model satisfiable. Its use is further restricted to instances for which the unsatisfiability is detected in the root node.

Korikov and Beck [12] introduced a quadratic programming algorithm for computing counterfactual explanations based on the previous work of Wang [33]. The algorithm takes any user's question that can be represented by linear or

quadratic constraints. Counterfactual explanations [32] answer questions of the form "Why this solution and not some other solution q?" They then show to the user that with some changes to the input, the optimal solution can be q.

3 Methodology

In the following, we propose *CMPCS-Finder* to provide corrections for an unsatisfiable instance that are understandable by a user. The corrections are modifications to the parameters of the instance that can restore feasibility. A CMPCS consists of a subset of parameters and their corrections to make the instance satisfiable at minimal cost. We represent a CMPCS as a set of equations of the form `<parameter>' = <original value>[+|-]<correction>`. The cost can be measured by the number of parameters to change or by any function expressing the amount of effort required to change the input. Our approach needs not only to solve the underlying NP-Hard CSP, but also to find a *Cost-Minimal Parameter Correction Subset* (CMPCS) to repair the unsatisfiability and provide insight into the underlying cause. Correcting the unsatisfiability is, in itself, computationally harder than solving the original problem. Therefore, we present a general approach that uses MCS reductions and is later refined in the hope of reducing the total computation time. *CMPCS-Finder* is summarized in Figure 1. It starts with an unsatisfiable instance, the existing hard model, and the instance (user) data. The *Soft Model* is created by the expert with, optionally, an MCS reduction model. The correctable parameters subset is extracted from the unsatisfiable instance and injected into the *Soft Model*. Optionally, the subset can pass through the *MCS Reduction* to reduce its size. Then, using some settings for the objective function, the *Soft Model* outputs a CMPCS.

3.1 Softening the Parameters

When using *CMPCS-Finder*, the modeling expert defines a subset of *correctable parameters* $\mathcal{P}^{\text{cor}} \subseteq \mathcal{P}$ that can have their values corrected. The expert chooses the correctable parameters based on their understanding of the real-world application. These can be parameters of constraints, including unary constraints that define the domain of a variable. Parameters that are not defined as correctable can impact unsatisfiability in general, but since they are not selected by the modeling expert, they should not contribute to unsatisfiability in isolation. This approach is particularly useful for data-driven problems, which are common in various industries. Problems that are not data-driven (e.g., the N-Queen Problem) would not benefit from *CMPCS-Finder*.

We are interested in unsatisfiable instances where the values of correctable parameters are the cause of the unsatisfiability. Moreover, we are interested in non-trivial unsatisfiable instances. An instance is trivially unsatisfiable if its unsatisfiability can be detected by a simple algorithm known to the modeling expert, for example, by an assertion on the parameters.

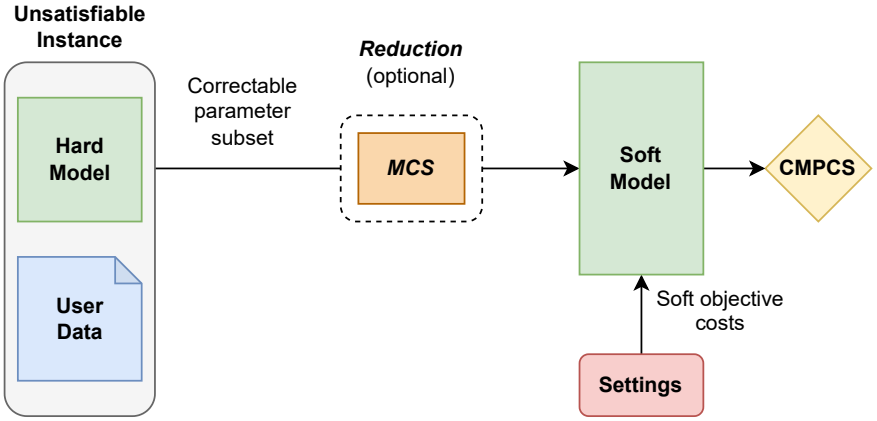


Fig. 1. High-level summary of *CMPCS-Finder*.

In order to compute a CMPCS, *CMPCS-Finder* transforms the preexisting hard model by replacing the correctable parameters with new variables called *parameter variables*. The resulting model is a *soft* model with the same constraints as the preexisting model. In the soft model, the objective function is replaced (or added, if the original problem is a satisfaction problem) by a cost function that minimizes the corrections needed to restore satisfiability. In practice, four forms of objective functions are of interest:

1. Minimizing the absolute differences between the parameter variables and their original parameter values;
2. Minimizing the number of parameter variables assigned to a different value than their original parameter value;
3. Minimizing objective 1, then minimizing objective 2;
4. Minimizing objective 2, then minimizing objective 1.

The first objective minimizes the amount of change to the parameters in order to obtain a satisfiable instance, while the second objective minimizes the number of parameters to change. It is possible to adapt the first objective by choosing a different distance measure between the parameter variable and its original value, such as the square of the difference. For the first and second objectives, it is also possible to choose a different aggregator, such as the weighted sum.

An example of a preexisting model, written in the MiniZinc modeling language [19], is shown in Figure 2a while the *soft model* is shown in Figure 2b. In the MiniZinc language, variables are declared with `var` and parameters with `par`. After the keyword, the integer type is declared with `int` or with a specific range of integer values (e.g., `0..100`). This corresponds to domain for variables or allowed values for parameters. MiniZinc will assert that the parameter's value

```
1 include "all_different.mzn";
2 var int: y;
3 par 0..100: a; par -20..20: b;
4
5 constraint y >= a;
6 constraint y < b;
7 constraint all_different([y, a, b]);
8
9 solve minimize y;
10 a = 10; b = 8; % User parameters
```

(a) Preexisting simple CSP model.

```
1 include "all_different.mzn";
2 var int: y;
3 par 0..100: a  ; par -20..20: b;
4 var 0..100: Xa; var -20..20: Xb; % New parameter variables
5
6 constraint y >= Xa;
7 constraint y < Xb;
8 constraint all_different([y, Xa, Xb]);
9
10 solve minimize Wa * abs(Xa - a) + Wb * abs(Xb - b);
11 a = 10; b = 8;                    % User parameters
12 par int: Wa = 2; par int: Wb = 1; % Objective weights
```

(b) Soft model of the CSP.

Fig. 2. Comparison of a simple CSP model and its soft version, in the MiniZinc modeling language, as created by *CMPCS-Finder*.

for a given instance is strictly inside the given range. The `constraint` and `solve minimize` keywords are respectively used for declaring constraints and the objective function. In this example, the `all_different` function is declared to use the solver-specific global constraint or a decomposition alternative if none exists. The correctable parameters a and b are defined on line 3 with allowed values $[0, 100]$ and $[-20, 20]$. In the preexisting model, these ranges serve as a check to ensure that they are assigned to correct values while importing the data. In the soft model, variables X_a and X_b replace the parameters used in the constraints. Their domains are set to the allowed values of the parameters, but one can sometimes restrict these variables further. The CMPCS found by *CMPCS-Finder* is $\{b' = 8 + 4\}$ which means the parameter b had a value of 8 and was corrected to $8 + 4 = 12$ to make the instance satisfiable. This CMPCS has a cost of 4. In our example, we can see that the smaller X_b is, the more constrained X_a is. In order for the solution to be relaxed, one necessarily needs to assign X_b to a value strictly greater than X_a can be. For that reason, the domain of X_b could be set directly to $[\min(\mathrm{dom}(a)) + 1, 20]$. These domain adjustments are problem-specific and need to be programmed by the expert, but they allow to reduce the search space.

Although, most parameters are easily converted into variables, there are situations where the conversion cannot be automatic. Some global constraints only accept constant parameters. Such constraints require to be encoded differently in the model to allow the correctable parameter to be turned into a variable. A parameter can define the domain of a variable, but a variable cannot define the domain of another variable. In such a situation, the domain of the variable can be encoded using parameterized unary constraints (e.g. $p_1 \leq X \leq p_2$). A parameter can define the length of an array of variables. To make this parameter correctable, the whole model might have to be revised. The domain of a variable associated with a correctable parameter can be copied from the given parame-

ter's allowed values. In industry, it is good practice to also restrict the values of parameters to avoid invalid data to reach the solver.

We implemented *CMPCS-Finder* using a custom compiler to create the soft model. The soft model is made from the original constraint model and some settings set by the modeling expert, such as the subset of *correctable parameters* or weights for the cost function. With these as inputs, our compiler can generate most parts of the soft model, accelerating the expert's job of providing an unsatisfiability resolving tool to the users.

Unlike traditional approaches in soft constraint literature [27, 28, 34], *CMPCS-Finder* allows constraints to be softened in multiple ways, which has two advantages. First, it eliminates the need to rewrite constraints to achieve a one-to-one relationship between a constraint and a parameter. This improves the readability of the model and simplifies the modeling expert's task. Second, this unlocks the use of many global constraints, which often have multiple input parameters that can be replaced by variables. Thus, common global constraints (e.g. GCC, Cumulative, NValue, Knapsack, etc.) can be used directly instead of decomposing them, which allows the use of their powerful filtering algorithms.

3.2 Reducing the Parameter Variables

Solving the soft model to produce a CMPCS can take a long computation time due to the complexity of the problem. For many industrial problems, there are numerous parameters. This makes it hard for the solver to find a solution in a reasonable time, since the parameters are changed into variables. To reduce computation complexity at the expense of an optimal solution, we present the *MCS Reduction* technique that can be executed before solving the soft model. This technique reduces the number of parameters that can be softened by the solver, which we call a *reduced correctable parameter subset*. In other words, parameters that are not in this subset will not be replaced by parameter variables. The reduction approach can be executed in parallel while, in addition, solving the soft model without reduction. The system can report to the user the first CMPCS found by any technique.

MCS Reduction This reduction is inspired by the computation of an MCS. However, instead of finding a minimal subset of constraints to correct, we find a set of constraints such that the union of their associated correctable parameters is minimal. In other words, we want to find the set of constraints to correct that involves the least number of parameters. To do so, we use the following model.

$$\min \textstyle\sum_{p \in \mathcal{P}} \beta_p$$

$$\alpha_C \iff C \qquad \forall\, C \in \mathcal{C} \quad (1)$$

$$\alpha_C \vee \beta_p \qquad \forall C \in \mathcal{C}\,, \forall\, p \in P(C) \quad (2)$$

Constraints (1) reify every constraint $C \in \mathcal{C}$ with a Boolean variable α_C, indicating whether the constraint is satisfied or not. We associate a Boolean variable

β_p with each parameter $p \in \mathcal{P}$, indicating whether the parameter can belong to the correction set. Constraints (2) ensure that unsatisfied constraints have their parameters selected. The objective is to minimize the number of selected parameters. The parameters whose variable β_p is set to true are selected to be correctable parameters. Once the parameters are selected, we create the soft model as usual, but only the selected correctable parameters are replaced by parameter variables. Parameters that are not selected remain constant in the model.

According to the definition of an MCS, removing these constraints corrects the unsatisfiability. Since *CMPCS-Finder* focuses on the user data, we instead correct the parameters that are related to these constraints. This makes the correction more flexible than simply removing all constraints in the MCS. In the example from Figure 2, a possible MCS might be $\{y \geq a\}$, and the reduced subset of correctable parameters would be $\{a\}$. Using this reduced subset of correctable parameters in the soft model leads to a faster computation time and the new CMPCS $\{a' = 10 - 4\}$. The cost of this solution is 8, which is a higher cost than the CMPCS found without reduction (4).

This parameter reduction technique allows solving a soft model with fewer variables, reducing the instance complexity. However, this comes at the price that the CMPCS might not be optimal with respect to the whole set of correctable parameters. In fact, there is even the possibility that too few parameters are selected for correction, leading to a soft model that remains unsatisfiable.

To address this issue, there exist conditions that guarantee the reduction technique selects correctable parameters that make the instance satisfiable. One such condition is the existence of correctable parameter values that turn constraints into tautologies. This is the case for a constraint $X \geq p$, where the parameter p can be fixed to the smallest value in the domain of X.

Constraints that contain no correctable parameters are put in the background and cannot be deactivated. If the unsatisfiability is caused by background constraints, then the data has no error, but the model itself is incorrect.

4 Experiments

We implemented *CMPCS-Finder* in Python 3.12. The first component takes as input a model written in MiniZinc 2.8.5 [19] and outputs the soft model and the MCS reduction model. The second component uses the models to compute a CMPCS. *CMPCS-Finder* can use the MCS reduction model to compute the reduced set of correctable parameters. With the reduced set (`MCS-Reduction`) or the complete set (`No-Reduction`), *CMPCS-Finder* uses the soft model to compute a CMPCS.

In our experiments, *CMPCS-Finder* either used the solver Chuffed 0.13.2 [4] or OR-Tools CP-SAT 9.10.4067 [21], depending on the problem. All experiments were run on a 32-core Intel Xeon 4110 CPU@2.10GHz and 32 GB of memory, on Windows. We set a 100-second time limit for the full approach, i.e. reducing the parameter set and finding the CMPCS. We also set specific timing rules for

finding the reduced set of correctable parameters: we used the best reduced set found within 50s, and if no set was found in 50s, we let the solver run until it has found one or timed out.

We considered three combinatorial problems, the *Resource-Constrained Project Scheduling Problem* (RCPSP) [8, 23], the *Tank Allocation Problem* (TAP) [25], and the *Roster Shift Bool Problem* (RSBP) [30]. These problems were chosen because they have standard benchmarks involving multiple parameters. In the following, we provide a description of the problems and how we altered the instances to make them unsatisfiable.

4.1 Resource-Constrained Project Scheduling Problem

The RCPSP [8, 23] requires to schedule a set of tasks $\mathcal{T}$ executing on a set of cumulative resources $\mathcal{R}$. Each task $i \in \mathcal{T}$ has a duration p_i and a consumption $h_{i,r}$ over a resource $r \in \mathcal{R}$. Each resource $r \in \mathcal{R}$ has a constant capacity c_r. At any time, the sum of the resource consumption of the tasks that execute on the resource r must not exceed the resource capacity c_r. The tasks are subject to precedence constraints, where some tasks must complete before other tasks start. The objective is to minimize the makespan, i.e. the time to complete all the tasks. We implemented the model given in CSPLib [20] in MiniZinc.

The correctable parameters are the durations p_i, the resource consumptions $h_{i,r}$, and the resource capacities c_r. In real life, a task could be shortened by allowing resources to work overtime. For example, a 5-day task can take 4 days through nighttime labor [3, 5, 29]. Resource consumptions or resource capacities can be altered by temporarily hiring additional resources. We therefore have $\mathcal{P}^{\text{cor}} = \{p_i \mid i \in \mathcal{T}\} \cup \{h_{i,r} \mid i \in \mathcal{T}, r \in \mathcal{R}\}$. We assume that the precedences cannot be corrected. The *CMPCS-Finder*'s soft model creates a variable X for each correctable parameter. The objective function is:

$$\begin{aligned} \min\ & 2 \textstyle\sum_{i \in \mathcal{T}} |X_{p_i} - p_i| + \sum_{i \in \mathcal{T}, r \in \mathcal{R}} |X_{h_{i,r}} - h_{i,r}| \\ & + 5 \textstyle\sum_{r \in \mathcal{R}} |X_{c_r} - c_r| \end{aligned}$$

We randomly selected 10 instances from PSPLib [11] with 30, 60, 90, and 120 tasks. As a first step to make these instances unsatisfiable, we added a constraint for the makespan to be no greater than the pre-calculated optimal makespan. This preserves the feasibility of the instances, but reduces the solution space to optimal solutions. In Section 4.4, we explain how the correctable parameters are modified to make the instance unsatisfiable.

4.2 Tank Allocation Problem

The TAP [25] is a problem of allocation between a set of cargoes $\mathcal{F}$ and a set of tanks $\mathcal{T}$. Each tank $t \in \mathcal{T}$ has a capacity v_t and a set of neighboring tanks $\mathcal{N}_t \subset \mathcal{T}$. Each cargo $f \in \mathcal{F}$ has a load q_f, a set of incompatible tanks $\mathcal{U}_f \subset \mathcal{T}$, and a set of incompatible cargoes $\mathcal{O}_f \subset \mathcal{F}$. One needs to assign each cargo to one or multiple compatible tanks. We simplify the problem by maximizing the

number of unallocated tanks. We use the model provided in CSPLib [24] that we implemented in MiniZinc.

The correctable parameters are the cargo loads and the tank capacities, i.e. $\mathcal{P}^{\text{cor}} = \{q_f \mid f \in \mathcal{F}\} \cup \{v_t \mid t \in \mathcal{T}\}$. We do not consider tank and cargo compatibility as a correctable parameter nor the neighboring relations. The *CMPCS-Finder*'s soft model creates a variable X for each correctable parameter. The objective function is:

$$\min \sum_{t \in \mathcal{T}} |X_{v_t} - v_t| + 2 \sum_{f \in \mathcal{F}} |X_{q_f} - q_f|$$

We generated 10 instances using the instance generator available in CSPLib. We ran the scripts using $|\mathcal{F}| \in \{5, 10, 15\}$, $|\mathcal{T}| \in \{10, 15, 25\}$. We added the objective function as a constraint so the instances remain satisfiable, but its solution space is restricted to optimal solutions. Finally, we altered some correctable parameters as described in Section 4.4.

4.3 Roster Shift Bool Problem

The RSBP is an assignment problem that is part of the 2023 MiniZinc Challenge [30]. There is a set of employees $\mathcal{E}$ that need to be assigned to a set of shifts $\mathcal{S}$. Each employee $e \in \mathcal{E}$ has a number of hours c_e associated to the length of their contract. An employee e also has a set of skill levels $\mathcal{R}_e = \{1, 2, \ldots, l_e\}$. Each shift $s \in \mathcal{S}$ has a start time start_s, an end time end_s, and a required skill level req_s. A binary matrix variable $A_{e,s}$ is used to choose if an employee e works the shift s.

The correctable parameters are the start times, the end times, the skills required for a shift, and the contracts, i.e. $\mathcal{P}^{\text{cor}} = \{\text{start}_s \mid s \in \mathcal{S}\} \cup \{\text{end}_s \mid s \in \mathcal{S}\} \cup \{\text{req}_s \mid s \in \mathcal{S}\} \cup \{c_e \mid e \in \mathcal{E}\}$. The employees' skills are not correctable. The *CMPCS-Finder*'s soft model has a variable X for each correctable parameter and the following objective function:

$$\begin{aligned} \min\ & 2 \sum_{e \in \mathcal{E}} |X_{c_e} - c_e| + \sum_{s \in \mathcal{S}} |X_{\text{start}_s} - \text{start}_s| \\ & + \sum_{s \in \mathcal{S}} |X_{\text{end}_s} - \text{end}_s| + 3 \sum_{s \in \mathcal{S}} |X_{\text{req}_s} - \text{req}_s| \end{aligned}$$

We used the "large.dzn" instance provided by the MiniZinc Challenge to craft 10 smaller satisfiable instances. For each instance, we selected a subset of employees and shifts from the large instance. The original instance has $|\mathcal{S}| = 297$ shifts and $|\mathcal{E}| = 25$ employees. Our instances have 10, 15, or 25 shifts with 5, 8, or 10 employees. To make the instances unsatisfiable, we first solved the original instance and let d be the objective value, i.e. the cumulative difference of hours between the employees' contracts and their assigned hours. We replaced the objective function with the constraint:

$$\sum_{e \in \mathcal{E}} \left| c_e - \sum_{s \in \mathcal{S}} (\text{end}_s - \text{start}_s) A_{e,s} \right| \leq d.$$

The instance remains satisfiable, but the solution space becomes restricted. We then altered some correctable parameters as explained in the next section.

4.4 Creating Unsatisfiable Instances

We want to create a benchmark of unsatisfiable instances, which are not "trivial" to explain. For example, with the RCPSP, a polynomial-time algorithm could detect whether the precedences form a cycle forcing, by transitivity, a task to be its own predecessor. We want to construct unsatisfiable instances that are close to feasible ones, which is useful for comparing our corrections.

We created a generator of unsatisfiable instances that takes as input a satisfiable instance. It randomly selects three correctable parameters. It modifies each selected parameter p according to the formula $p' := p + \Delta_p$, where Δ_p is drawn uniformly from a given range. The modified instance is submitted to the solver for a satisfiability test. If the instance is still satisfiable (or is not proven to be unsatisfiable after five minutes), the generator restarts from the original instance by choosing three new parameters and three new modifications. In order to help this process converge to an infeasible instance, both bounds of the range are updated according to the formula $x' := x^{\left(1+\frac{i}{8.25}\right)}$ where x is the original range bound and i is the iteration number, starting with $i = 1$. In other words, the generator allows modifications to the original parameters to be larger every failed iteration.

As explained in previous sections, each problem has 10 original satisfiable instances. Using our generator, we generated five unsatisfiable instances for every original instance, for a total of 50 unsatisfiable instances for each problem[3].

For each instance, we saved the three parameters that were altered by the generator. These parameters may not form a CMPCS, as their cost might not be minimal. Indeed, maybe only one of the three parameter changes made the instance unsatisfiable, or maybe a parameter was increased by 10 units, while only 8 units were sufficient. For experimental comparison, we define `Oracle` as a reduction technique that considers these three parameters as the reduced correctable parameter subset. Thus, this oracle uses the soft model to compute a CMPCS based on knowledge from the unsatisfiable instance generation.

4.5 Results

Qualitative Evaluation We first proceed to a qualitative evaluation of our approach by showing some CMPCSs computed by the *CMPCS-Finder* with `No-Reduction` for the RCPSP. Recall that CMPCS is a set of equations of the form `<parameter>' = <original value>[+|-]<correction>`. For instance, the CMPCS returned for instance 2_j3010_7 is $\{p'_8 = 3 - 2\}$, which means that the duration of task 8 was 3 units of duration and was corrected to $3 - 2 = 1$ to make the instance feasible. This CMPCS has a cost of 4, since the duration parameter has a weight of 2 in the objective function, and $2 \cdot |-2| = 4$. The format we use makes it relatively easy for a program to output the CMPCS in natural language, for example as "the duration of task 8 should be decreased by 2."

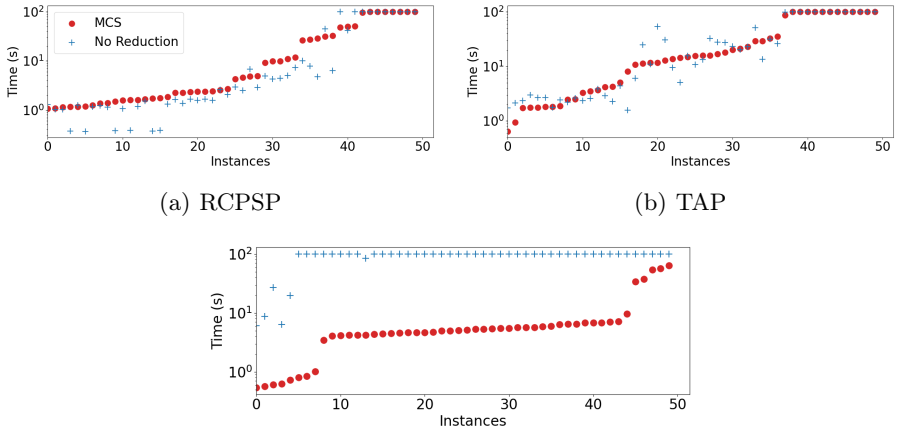
[3] Generator values, models, and instances are available at https://github.com/AntoineLaviolette1/cmpcs-finder.

Computing a single CMPCS might not be sufficient for a human to fully understand why an instance is unsatisfiable. In our example, other tasks than task 8 could have their duration reduced to fix the instance. Here are two other CMPCS whose cost 4 is also optimal: $\{p'_{29} = 6 - 1, h'_{19,3} = 3 - 2\}$ and $\{p'_{29} = 6 - 2, h'_{16,3} = 2 - 1, h'_{19,3} = 3 - 1\}$. We can see that the infeasibility arises from a shortage of resource 3 with the combination of task precedences, packing the tasks in a tight time window. The three CMPCSs propose to reduce the consumption of resource 3, either by reducing the duration of the tasks that require this resource, or by reducing the consumption of the tasks of this resource. In this case, the user can choose any proposed CMPCS or decide to make other corrections to solve the problem with resource 3. Here are the corrections that would revert the instance to its original form: $\{p'_{24} = 11 - 4, h'_{26,3} = 12 - 3, c'_2 = 22 + 2\}$ The cost of this correction is $2 \cdot |-4| + |-3| + 5 \cdot 2 = 21$. Using the `Oracle` technique to reduce the correctable parameters to $\{p_{24}, h_{26,3}, c_2\}$, *CMPCS-Finder* finds the CMPCS $\{h'_{26,3} = 12 - 7\}$ that has a cost of 7, slightly worse than the three CMPCSs found by `No-Reduction`. In this case, *CMPCS-Finder* obtains a lower cost, since it does not use a reduced correctable parameter subset. The original instance alterations are consistent with our explanations: the consumption of resource 3 needs to be reduced and the duration of task 24, that consumes resource 3, also needs to be reduced. Finding multiple CMPCSs or *Near-Cost-Minimal* Parameter Correction Subsets can provide different valid explanations that a user, without any knowledge of the underlying model, can understand and draw conclusions from.

MCS Comparison We compare our CMPCS approach against an MCS, since both approaches find corrections to restore feasibility. We extract an MCS from the same unsatisfiable instance (2_j3010_7) as in our previous evaluation. The MCS returned is Cumulative(3). Therefore, removing the Cumulative constraint associated with resource 3 makes the instance satisfiable. This is a worse correction than what *CMPCS-Finder* finds. In both cases, resource 3 is identified as problematic. But the MCS method only provides one disruptive correction that is, in many cases, impractical. The *CMPCS-Finder* finds a smaller and less disruptive correction, i.e., to reduce the duration of task 8 by 2 units.

Quantitative Evaluation Figure 3 compares the solving times (log scale) of `No-Reduction` and `MCS-Reduction`. The instances are sorted by the computation time of the `MCS-Reduction`. For the RCPSP, the `MCS-Reduction` does not improve the computation time. The solver is able to solve the soft model with a full set of correctable parameters. The `MCS-Reduction` provides computation times that are still reasonable. For the TAP, there is no clear faster technique. For the RSBP, the `MCS-Reduction` is the fastest way to solve this problem, by far. This is not an anomaly, since for every instance of RSBP, the `MCS-Reduction` was much faster.

Figure 4 shows the cost of the CMPCS for the two techniques and `Oracle`. The instances are sorted by the `MCS-Reduction`'s cost. `Oracle` globally obtains



(a) RCPSP (b) TAP

(c) RSBP

Fig. 3. Graphs comparing the computation time for `MCS-Reduction` and `No-Reduction`.

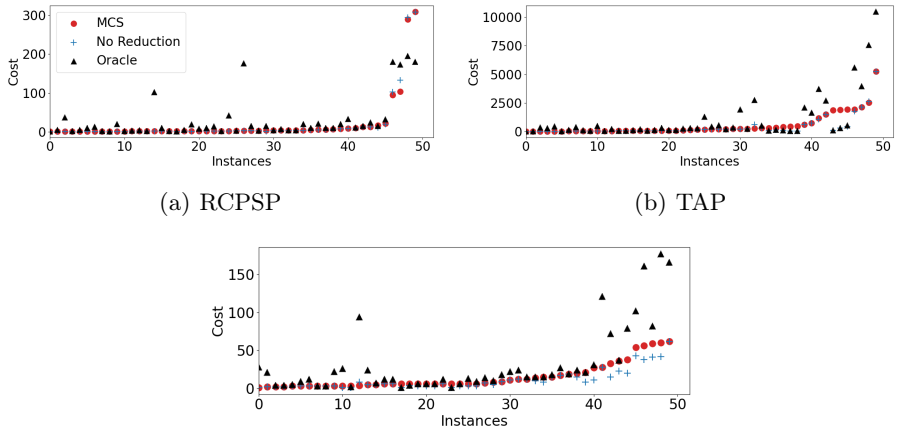
CMPCS with the highest cost. We also observe that the `MCS-Reduction` increases sometimes by a small amount the cost of the CMPCS, compared to `No-Reduction`. This is more frequent with RSBP, the harder problem. The `No-Reduction` technique finds the CMPCS with the lowest cost, but this is not always the case because of the timeout. The `MCS-Reduction` found a lower cost for 3 instances on the RCPSP, 2 on the TAP and 1 on the RSBP.

Considering both figures, for problems that take longer to compute a CMPCS, such as RSBP, the `MCS-Reduction` technique can be useful for its quick computation time. The `No-Reduction` technique appears to generally take the same amount of time and can produce better CMPCS for easier problems.

Table 1. Comparison of the techniques by problem.

Problem	Technique	Optimal	Cor par.
RCPSP	`No-Reduction`	40	314.00
	`MCS-Reduction`	43	59.04
TAP	`No-Reduction`	38	31.00
	`MCS-Reduction`	38	21.58
RSBP	`No-Reduction`	6	57.60
	`MCS-Reduction`	50	2.46

In Table 1, column **Optimal** indicates the number of CMPCS that were proven optimal. For all instances, the solver found at least a solution (a CMPCS), but not all proven optimal. The column **Cor par.** gives the average number of



(a) RCPSP (b) TAP

(c) RSBP

Fig. 4. Graphs comparing the cost of the CMPCS for `MCS-Reduction`, `No-Reduction`, and `Oracle`.

correctable parameters used. The `MCS-Reduction` significantly reduces the number of correctable parameters. Our results show only small differences in average computation times with smaller correctable parameter subsets for RCPSP and TAP. An average of 20 to 60 correctable parameters yields too many combinations for *CMPCS-Finder* to find a solution quickly. The time taken to compute the reduction is equivalent to the time saved when solving a smaller soft model. For the RSBP, there are few parameters, which accelerates the computation.

5 Conclusion

We introduced *CMPCS-Finder*, a data-oriented method for restoring feasibility in unsatisfiable constraint problems by identifying minimal changes to user-provided parameters. By formalizing *Cost-Minimal Parameter Correction Subsets* (CMPCS) and generating soft models in which parameter values become optimization variables, the approach focuses directly on suggesting concrete corrections that users can apply to their data. We proposed a reduction technique to limit the number of parameters considered, which can accelerate solving depending on the problem structure. Experiments on RCPSP, TAP, and RSBP instances showed the relevance of this approach and demonstrated that, in most cases, it successfully identifies low cost changes in the data.

Future research can use the concept of a parameter-driven approach instead of the traditional constraint-driven approach to create not only corrections but also explanations that users can understand without a computer science background.

References

1. Bleukx, I., Devriendt, J., Gamba, E., Bogaerts, B., Guns, T.: Simplifying stepwise explanation sequences. In: Proceedings of the 29th International Conference on Principles and Practice of Constraint Programming (CP 2023). pp. 11:1–11:20 (2023)
2. Bogaerts, B., Gamba, E., Guns, T.: A framework for step-wise explaining how to solve constraint satisfaction problems. Artificial Intelligence **300**, 103550 (November 2021)
3. Boudreault, R., Simard, V., Lafond, D., Quimper, C.G.: A constraint programming approach to ship refit project scheduling. In: Proceedings of the 28th International Conference on Principles and Practice of Constraint Programming (CP 2022). pp. 10:1–10:16 (2022)
4. Chu, G.G.: Improving Combinatorial Optimization. Ph.D. thesis, The University of Melbourne (2011)
5. Cloutier, S., Quimper, C.G.: Cumulative Scheduling with Calendars and Overtime. In: Proceedings of the 30th International Conference on Principles and Practice of Constraint Programming (CP 2024). pp. 7:1–7:16 (2024)
6. Dev Gupta, S., Genc, B., O'Sullivan, B.: Explanation in constraint satisfaction: A survey. In: Proceedings of the 30th International Joint Conference on Artificial Intelligence. pp. 4400–4407 (2021)
7. Gamba, E., Bogaerts, B., Guns, T.: Efficiently explaining CSPs with unsatisfiable subset optimization. In: 29th International Joint Conference on Artificial Intelligence. pp. 1381–1388 (2021)
8. Hartmann, S., Briskorn, D.: An updated survey of variants and extensions of the resource-constrained project scheduling problem. European Journal of Operational Research **297**(1), 1–14 (February 2022)
9. Junker, U.: QUICKXPLAIN: Preferred explanations and relaxations for over-constrained problems. In: Proceedings of the 19th National Conference on Artifical Intelligence (AAAI). pp. 167–172 (2004)
10. Jussien, N., Ouis, S.: User-friendly explanations for constraint programming. In: Proceedings of the 11th Workshop on Logic Programming Environments (WLPE'01) (2001)
11. Kolisch, R., Sprecher, A.: PSPLIB - A project scheduling problem library. European Journal of Operational Research **96**(1), 205–216 (January 1997)
12. Korikov, A., Beck, J.C.: Objective-based counterfactual explanations for linear discrete optimization. In: Proceedings of the 20th International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2023). pp. 18–34 (2023)
13. Lauffer, N., Topcu, U.: Human-understandable explanations of infeasibility for resource-constrained scheduling problems. In: ICAPS 2019 Workshop XAIP (April 2019)
14. Leo, K., Tack, G.: Debugging unsatisfiable constraint models. In: Proceedings of the 14th International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2017). pp. 77–93 (2017)
15. Liffiton, M.H., Malik, A.: Enumerating infeasibility: Finding multiple MUSes quickly. In: Gomes, C., Sellmann, M. (eds.) Proceedings of the 10th International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2013). pp. 160–175 (2013)

16. Liffiton, M.H., Previti, A., Malik, A., Marques-Silva, J.: Fast, flexible MUS enumeration. Constraints **21**(2), 223–250 (April 2016)
17. Marques-Silva, J., Mencía, C.: Reasoning about inconsistent formulas. In: Proceedings of the 29th International Joint Conference on Artificial Intelligence. pp. 4899–4906 (2020)
18. Nadel, A., Ryvchin, V., Strichman, O.: Accelerated deletion-based extraction of minimal unsatisfiable cores. Journal on Satisfiability, Boolean Modeling and Computation **9**, 27–51 (November 2014)
19. Nethercote, N., Stuckey, P.J., Becket, R., Brand, S., Duck, G.J., Tack, G.: MiniZinc: Towards a standard CP modelling language. In: Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming (CP 2007). pp. 529–543 (2007)
20. Nightingale, P., Demirović, E.: CSPLib problem 061: Resource-constrained project scheduling problem (RCPSP) (1999)
21. Perron, L., Furnon, V.: OR-Tools. Google (2024), website: https://developers.google.com/optimization/
22. Povéda, G., Boumazouza, R., Strahl, A., Hall, M., Quintana-Amate, S., Alvarez, N., Bleukx, I., Tsouros, D., Verhaeghe, H., Guns, T.: Trustworthy and explainable decision-making for workforce allocation (2024), https://arxiv.org/abs/2412.10272
23. Pritsker, A.A.B., Waiters, L.J., Wolfe, P.M.: Multiproject scheduling with limited resources: A zero-one programming approach. Management Science (September 1969)
24. Schaus, P.: CSPLib problem 051: Tank allocation (1999)
25. Schaus, P., Régin, J.C., Van Schaeren, R., Dullaert, W., Raa, B.: Cardinality Reasoning for Bin-Packing Constraint: Application to a Tank Allocation Problem. In: Proceedings of the 18th International Conference on Principles and Practice of Constraint Programming (CP 2012. pp. 815–822 (2012)
26. Schiendorfer, A., Knapp, A., Anders, G., Reif, W.: MiniBrass: Soft constraints for MiniZinc. Constraints **23**(4), 403–450 (October 2018)
27. Senthooran, I., Klapperstueck, M., Belov, G., Czauderna, T., Leo, K., Wallace, M., Wybrow, M., de la Banda, M.G.: Human-centred feasibility restoration. In: Proceedings of the 27th International Conference on Principles and Practice of Constraint Programming (CP2021). pp. 203–243 (2021)
28. Senthooran, I., Klapperstueck, M., Belov, G., Czauderna, T., Leo, K., Wallace, M., Wybrow, M., Garcia de la Banda, M.: Human-centred feasibility restoration in practice. Constraints **28**(2), 203–243 (June 2023)
29. Smith, S.F., Fox, M.S., Ow, P.S.: Constructing and maintaining detailed production plans: Investigations into the development of knowledge-based factory scheduling systems. AI Magazine **7**(4), 45–61 (1986)
30. Stuckey, P.J., Feydy, T., Schutt, A., Tack, G., Fischer, J.: The MiniZinc Challenge 2008–2013. AI Magazine **35**(2), 55–60 (June 2014)
31. van Loon, J.N.M.: Irreducibly inconsistent systems of linear inequalities. European Journal of Operational Research **8**(3), 283–288 (November 1981)
32. Wachter, S., Mittelstadt, B., Russell, C.: Counterfactual explanations without opening the black box: automated decisions and the gdpr. Harvard Journal of Law & Technology **31**(2), 841–887 (2017)
33. Wang, L.: Cutting plane algorithms for the inverse mixed integer linear programming problem. Operation Research Letters **37**(2) (2009)
34. Yang, J.: Infeasibility resolution based on goal programming. Computers & Operations Research **35**(5), 1483–1493 (May 2008)